

Khaula Molapo

20240001

Artificial Intelligence

Week: 9

1. Scenario Analysis

A machine learning model can be used in an IT project to detect fraudulent transactions in a banking system. The model can be trained using historical transaction data containing examples of legitimate and fraudulent activities. The data may include information such as transaction amount, location, time, device type, transaction frequency, and account activity. During training, the model learns patterns that are commonly associated with fraudulent behaviour. When a new transaction occurs, the model analyses its characteristics and predicts whether the transaction is likely to be legitimate or suspicious. If a transaction receives a high fraud probability, the system can flag it for further investigation or require additional verification from the customer. The model can help financial institutions process large numbers of transactions faster than manual checking. However, the model must be trained with reliable data and regularly monitored because fraud patterns can change over time. Human review can also be used for important decisions.

2. Concept Research

Supervised learning is a machine learning approach in which a model learns from a dataset containing input examples and their correct outputs, known as labels. During training, the model identifies patterns between the input features and the target labels. After training, it can use these learned patterns to make predictions about new data. Supervised learning is commonly used for classification and regression tasks. For example, an IT system can use supervised learning to classify emails as spam or legitimate, predict equipment failures, or detect potentially fraudulent transactions. The quality of the training data is important because inaccurate, incomplete, or biased data can reduce the model's performance. Common supervised learning algorithms include decision trees, linear regression, logistic regression, support vector machines, and some ensemble methods.

3. Tool Practice

For the tool practice, I created a simple machine learning model using scikit-learn to classify data. The process involved preparing the dataset, separating the input features from the target values, splitting the data into training and testing sets, and training a classification model. The test data was then used to evaluate how well the model

performed on information it had not seen during training. I observed that scikit-learn makes machine learning easier because it provides ready-made tools for data splitting, model training, prediction, and evaluation. The activity helped me understand that building an ML model involves more than simply writing an algorithm. The data must be prepared correctly and the results must be evaluated. I also learned that a model can perform well on training data but still perform poorly on new data if it does not generalize properly.

4. Application Practice

An ML workflow describes the main stages followed when developing and using a machine learning solution. A basic workflow begins by defining the problem and collecting relevant data. The data is then cleaned and prepared before being divided into training and testing sets. Next, an appropriate machine learning algorithm is selected and trained using the training data. The model is then evaluated using test data and suitable performance measures. If the results are acceptable, the model can be deployed as part of an IT application. After deployment, its performance should be monitored because new data and changing conditions can affect its accuracy. The workflow helps organize the development process and reduces the chance of skipping important steps. It also makes it easier to identify problems with data, model performance, or deployment. A clear workflow therefore supports the development of reliable and maintainable ML applications.

Example scikit-learn implementation:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X = [[10], [20], [30], [40], [50], [60]]
y = [0, 0, 0, 1, 1, 1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
print(accuracy_score(y_test, predictions))
```